



第3章 组合逻辑电路

组合逻辑电路：电路在任一时刻的输出状态仅由该时刻的输入信号决定，与电路在此信号输入之前的状态无关。





3.2 组合逻辑电路的分析

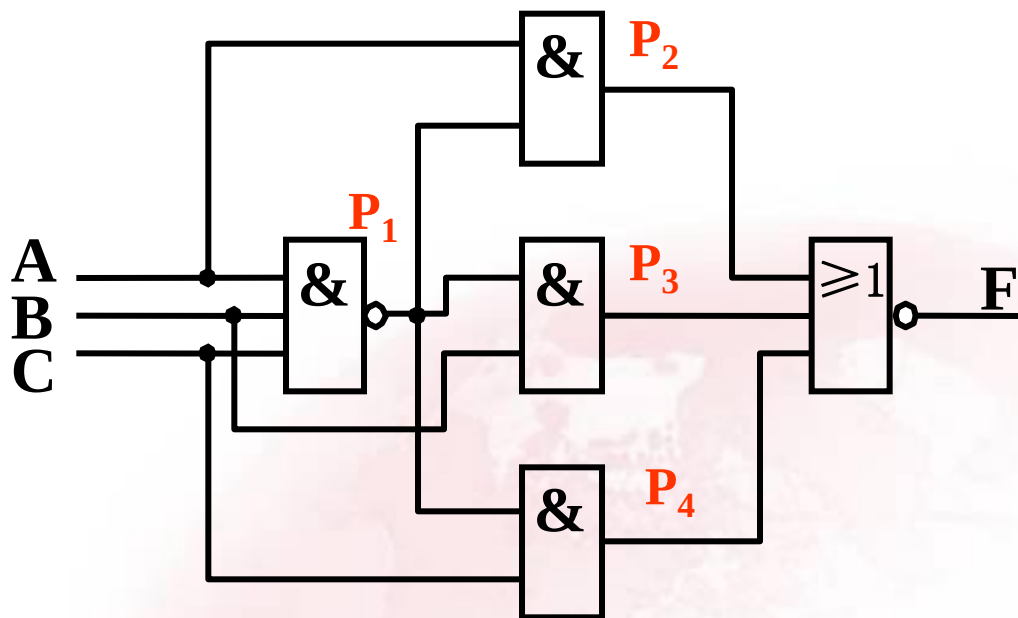
3.2.1 分析方法

分析步骤:

- (1) 根据**逻辑电路图**，写出输出逻辑函数**表达式**；
- (2) 根据**逻辑表达式**，列出**真值表**；
- (3) 由**真值表或表达式**分析**电路功能**。



例：分析下图所示逻辑电路



$$F = \overline{P_2 + P_3 + P_4}$$

$$P_2 = A \cdot P_1 \quad P_3 = B \cdot P_1 \quad P_4 = C \cdot P_1$$

$$P_1 = \overline{ABC}$$

$$\text{故: } F = \overline{(A+B+C) \cdot \overline{ABC}} = \overline{ABC} + \overline{\overline{ABC}} = \overline{ABC} + ABC$$

真值表

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

逻辑功能：一致电路



3.3 组合逻辑电路设计

一般步骤:

- (1) 由实际逻辑问题列出真值表;
- (2) 由真值表写出逻辑表达式;
- (3) 化简、变换输出逻辑表达式;
- (4) 画出逻辑图。



例：试用与非门设计一个三变量表决电路（不允许弃权），表决规则为少数服从多数。

解：(1) 列真值表

设：由A、B、C表示三个输入变量，F表示表决结果。并设A、B、C为“1”表示赞成，为“0”表示反对；F为“1”表示表决通过，为“0”表示不通过。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



(2) 化简、求最简函数表达式

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

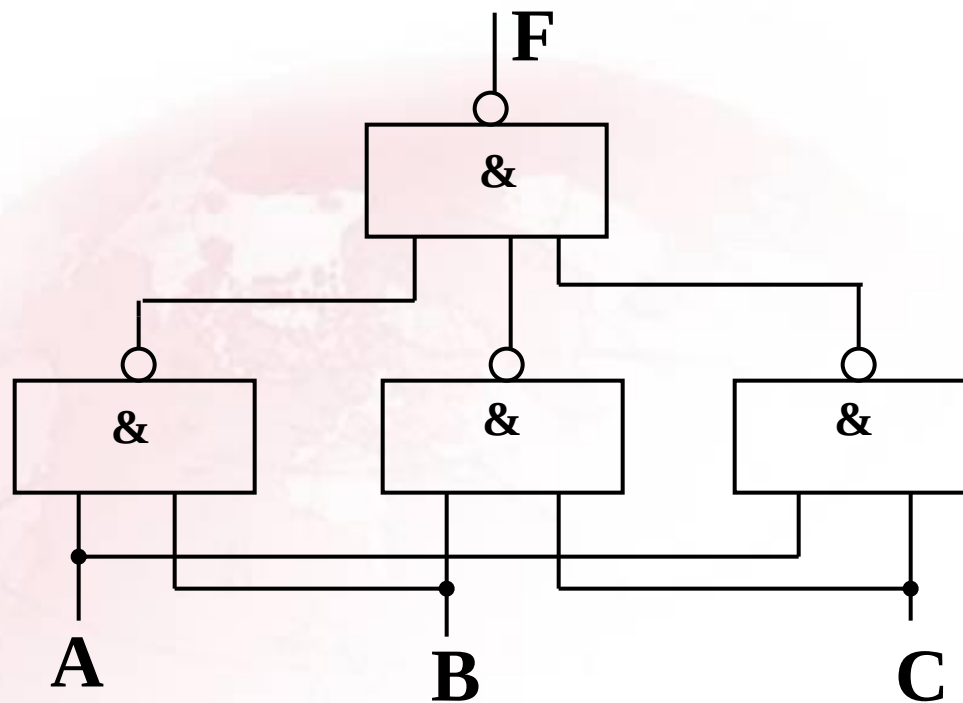
BC A	00	01	11	10
0			1	
1		1	1	1

$$F = AB + AC + BC$$

$$= \overline{\overline{A}\overline{B}} \cdot \overline{\overline{A}\overline{C}} \cdot \overline{\overline{B}\overline{C}}$$



(3) 画电路图





例：设计一个两位二进制数比较器。

解：设被比较的数分别为 $A=A_1A_0$ ， $B=B_1B_0$ ，

比较的结果用三个变量 F_1 、 F_2 、 F_3 ，

$A_1A_0 > B_1B_0$ 时，输出 $F_1=1$ ；

$A_1A_0 = B_1B_0$ 时，输出 $F_2=1$ ；

$A_1A_0 < B_1B_0$ 时，输出 $F_3=1$ 。



列真值表:

A_1	A_0	B_1	B_0	F_1	F_2	F_3	A_1	A_0	B_1	B_0	F_1	F_2	F_3
0	0	0	0	0	1	0	1	0	0	0	1	0	0
0	0	0	1	0	0	1	1	0	0	1	1	0	0
0	0	1	0	0	0	1	1	0	1	0	0	1	0
0	0	1	1	0	0	1	1	0	1	1	0	0	1
0	1	0	0	1	0	0	1	1	0	0	1	0	0
0	1	0	1	0	1	0	1	1	0	1	1	0	0
0	1	1	0	0	0	1	1	1	1	0	1	0	0
0	1	1	1	0	0	1	1	1	1	1	0	1	0



画卡诺图化简：

$A_1A_0 \backslash B_1B_0$		F_1			
		00	01	11	10
00					
01					
11	1				
10	1	1			1

$$F_1 = A_1 \bar{B}_1 + A_1 A_0 \bar{B}_0 + A_0 \bar{B}_1 \bar{B}_0$$

$A_1A_0 \backslash B_1B_0$		F_2			
		00	01	11	10
00	1				
01		1			
11				1	
10					1

$$F_2 = \bar{A}_1 \bar{A}_0 \bar{B}_1 \bar{B}_0 + \bar{A}_1 A_0 \bar{B}_1 B_0 \\ + A_1 \bar{A}_0 B_1 \bar{B}_0 + A_1 A_0 B_1 B_0$$



$A_1A_0 \backslash B_1B_0$		F_3			
		00	01	11	10
00			1	1	1
01				1	1
11					
10				1	

由 F_1 、 F_2 和 F_3 表达式可方便地用门电路实现比较器的逻辑功能。

$$F_3 = \bar{A}_1 B_1 + \bar{A}_1 \bar{A}_0 B_0 + \bar{A}_0 B_1 B_0$$



3.4 组合逻辑电路中的冒险

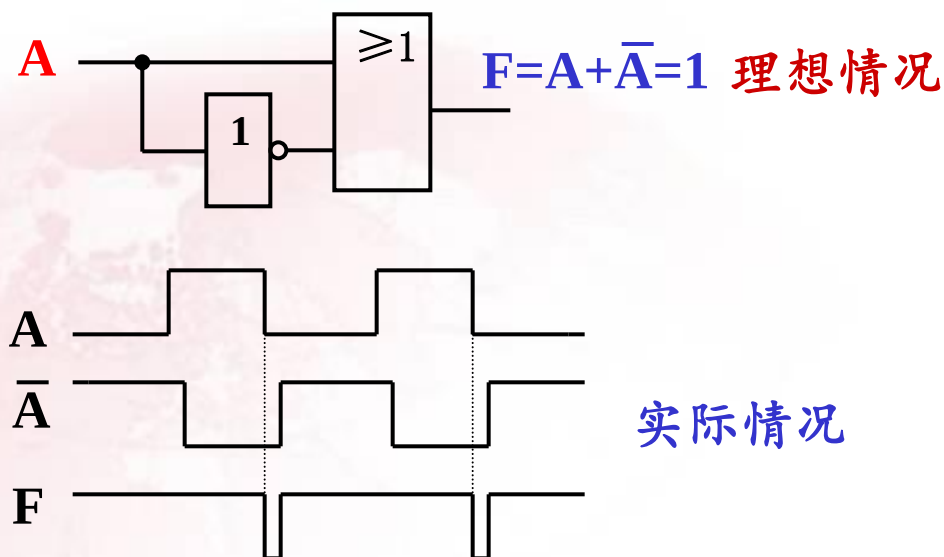
前面分析组合逻辑电路时，没有考虑电路中连线及逻辑门的延迟时间，而且默认电路中多个信号发生变化都是同时完成的。

实际上，信号经过导线及门电路的传输都需要一定的响应时间，多个信号发生变化时不可能完全同时等，这些实际因素，可能会使逻辑电路产生错误输出。通常把这种现象称为**竞争冒险**。



产生冒险的原因

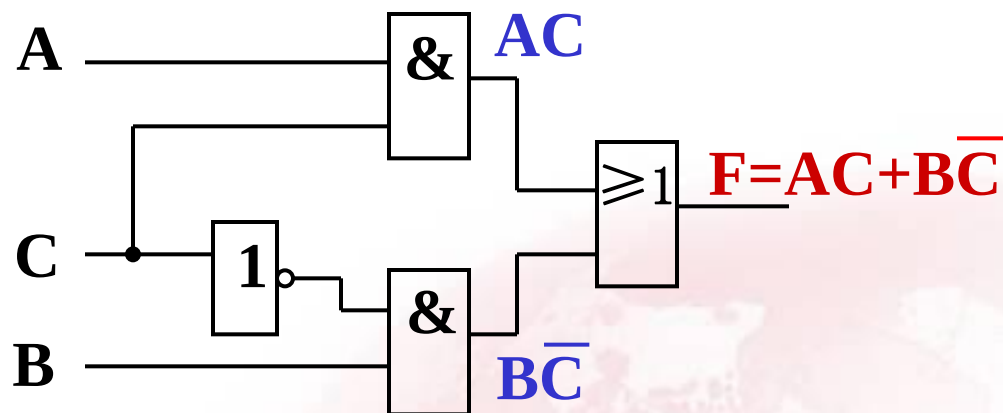
以例说明



造成冒险的原因是由于 A 和 $\bar{A}$ 到达或门的时间不同。



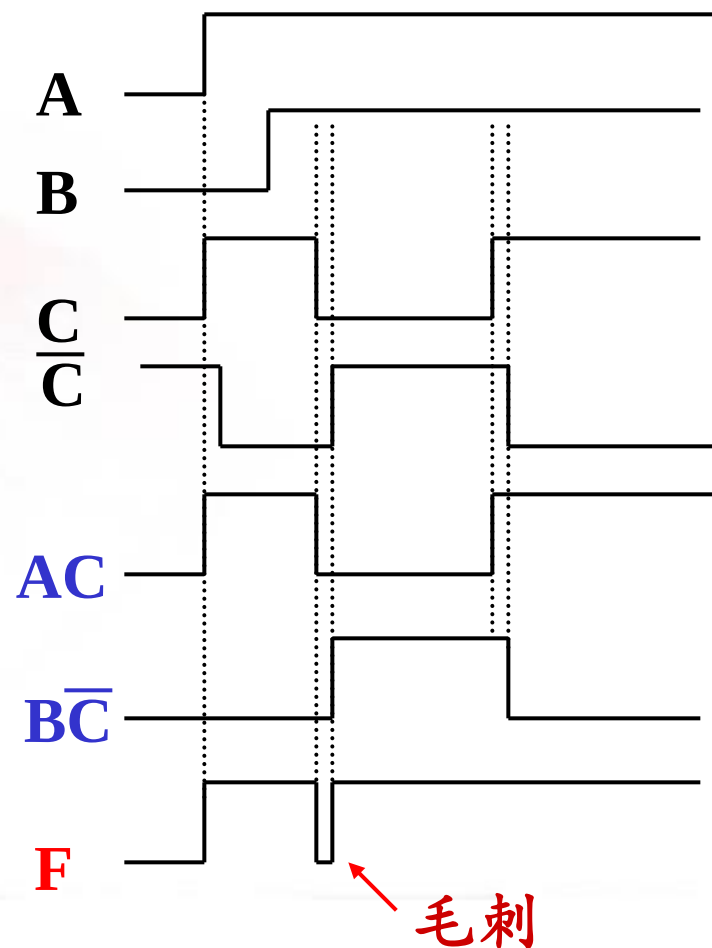
再举一例



(分析中略去与门和或门的延时)

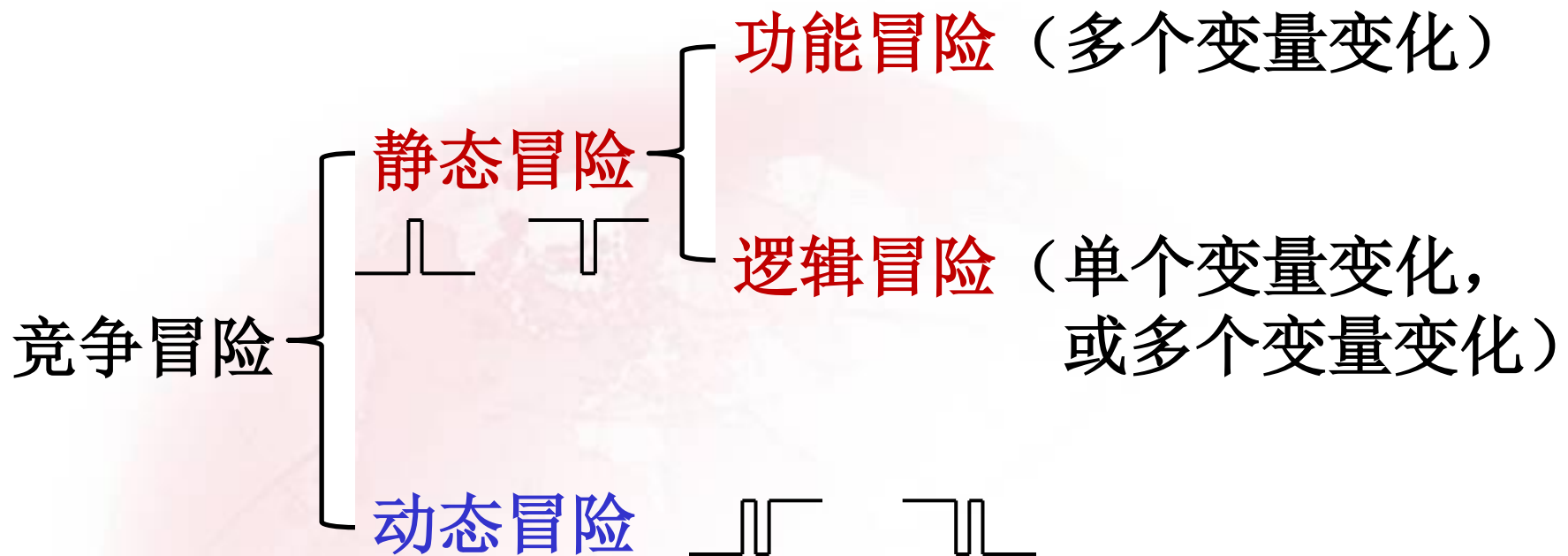
产生冒险的原因之一：

电路存在由非门产生的互补信号，且互补信号的状态发生变化时有可能出现冒险现象。





竞争冒险种类：





1. 静态冒险

(1) 静态功能冒险

当有多个变量发生变化时，稳态输出不应该发生变化，但实际输出产生了毛刺。

例：

A \ BC	BC			
	00	01	11	10
0	0	1	1	0
1	0	0	1	1

$ABC=001 \rightarrow 111$, $F=1$

如果B变化快于A，则出现011过渡状态，输出仍为1，故不会出现冒险。

但如果A变化快于B，则出现101过渡状态，输出为0。因此在输入变量变化过程中，输出出现 $1 \rightarrow 0 \rightarrow 1$ 的毛刺。



(2) 静态逻辑冒险

单个变量变化时，稳态输出不应该发生变化，但实际输出产生了毛刺。

P 个变量 ($P > 1$) 变化时，对于 2^P 个取值组合的输出全为0或1，即电路已经排除功能冒险，但输入变量发生变化时，电路仍有毛刺。

可见，静态逻辑冒险是在输入变量发生变化时，排除了功能冒险之后，由于门的延迟不同，而产生的冒险现象。



2. 竞争冒险的消除方法

逻辑冒险可以用修改逻辑设计的方法消除；

功能冒险不能用修改逻辑设计的方法消除。

(1) 修改逻辑设计

① 发现并消去互补变量

例如： $F=(A+B)(\bar{A}+C)$ ，在 $B=C=0$ 时， $F=A\bar{A}$ 。若直接根据这个逻辑表达式组成电路，就可能出现冒险。

将上式写成： $F=AC+\bar{A}B+BC$ ，已将 $A\bar{A}$ 去掉，则不会出现冒险。



② 增加乘积项

例如： $F=AC+B\bar{C}$ ，当 $A=B=1$ 时， $F=C+\bar{C}$ 。若直接根据这个逻辑表达式组成电路，就可能出现冒险。

将上式写成： $F=AC+B\bar{C}+AB$ ，这样，当 $A=B=1$ 时，不会出现 $F=C+\bar{C}$ ，所以 C 状态的变化，不会影响输出。

(2) 加选通脉冲

在数字系统中，可以用时钟信号的适当形式作为选通脉冲，消除竞争冒险。即**选通脉冲是在输入变化而引起电路的变化达到稳定后出现的**，这样取出的是无冒险的稳态输出，从而消除竞争冒险。



(3) 接入滤波电容

如果逻辑电路在较慢速度下工作，为了消去冒险，可以在输出端并联一电容，其容量在几十到几百皮法之间，该电容和门的输出电阻构成RC低通网络，对窄脉冲起平滑作用。



3.5 硬件描述语言——Verilog

目前许多数字电路都是用可编程逻辑器件（**PLD**，Programmable Logic Device）来实现的。



硬件描述语言（**HDL**，Hardware Description Language）就是可以描述硬件电路的功能、信号连接关系及定时关系的语言。

主要有：**VHDL**、**Verilog HDL**等。

这里主要介绍**Verilog HDL**的基本知识。



1. Verilog的历史

- Verilog语言是1983年由GDA（Gateway Design Automation）公司的Phil Moorby首创的，之后Moorby又设计了Verilog-XL仿真器，Verilog-XL仿真器大获成功，也使得Verilog语言得到推广使用。
- 1989年，Cadence收购了GDA，1990年，Cadence公开发表了Verilog HDL，并成立了OVI组织专门负责Verilog HDL的发展。
- Verilog于1995年成为IEEE标准，称为IEEE Standard 1364-1995（Verilog-1995）。
- IEEE“1364-2001”标准（Verilog-2001）也获得通过，多数综合器、仿真器都已支持Verilog-2001标准。



2. Verilog程序的基本结构

例：三人表决电路的Verilog描述（数据流描述方式）

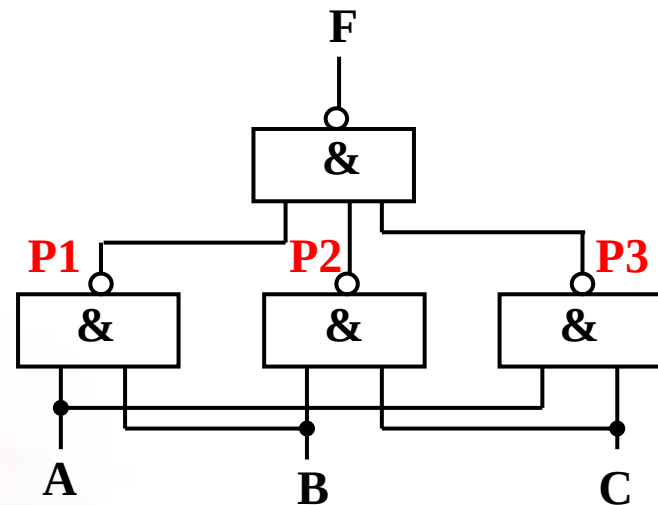
$$F = \overline{A}B + A\overline{B}C + \overline{A}BC$$

```

module vote1(A,B,C,F);    //模块名为vote1，端口列表A,B,C,F
    input A,B,C;           //模块的输入端口
    output F;              //模块的输出端口
    wire A,B,C,F;         //定义信号的数据类型
    assign F=~((~(A&B))&(~(B&C))&(~(A&C))); //逻辑功能描述
endmodule
    
```



例：三人表决电路的Verilog描述 (结构描述方式)



```
module vote2(A,B,C,F);
```

```
input A,B,C;
```

```
output F;
```

```
wire P1,P2,P3;
```

```
nand(P1,A,B);
```

```
nand(P2,B,C);
```

```
nand(P3,A,C);
```

```
nand(F,P1,P2,P3);
```

```
endmodule
```

//模块名为vote2，端口列表A,B,C,F

//模块的输入端口

//模块的输出端口

//电路内部节点声明

// “nand” 是Verilog内部定义的与非门



例：三人表决电路的Verilog描述 (行为描述方式)

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

```
module vote3(A,B,C,F); //模块名为vote3，端口列表A,B,C,F
    input A,B,C;        //模块的输入端口
    output F;           //模块的输出端口
    reg F;              //定义信号的数据类型
    always @ (*)
    begin
        case({A,B,C})   //用case语句完成电路功能的描述
            3'b000: F=0; 3'b001: F=0; 3'b010: F=0; 3'b011: F=1;
            3'b100: F=0; 3'b101: F=1; 3'b110: F=1; 3'b111: F=1;
            default: F=0;
        endcase
    end
endmodule
```



- Verilog程序是由模块构成的。每个模块的内容都嵌在 **module** 和 **endmodule** 两个关键字之间；每个模块实现特定的功能。
- 每个模块首先要进行端口定义，并说明输入和输出口（**input**、**output**或**inout**），然后对模块的功能进行定义。
- Verilog程序书写格式自由，一行可以写几个语句，一个语句也可以分多行写。
- 除了 **endmodule** 等少数语句外，每个语句的最后必须有分号。
- 可用 **/*.....*/** 和 **//.....** 对Verilog程序作注释。



(1) 模块声明

模块声明包括模块名字，模块输入、输出端口列表。

模块定义格式如下：

module 模块名(端口1， 端口2， 端口3，);



(2) 端口 (Port) 定义

对模块的输入输出端口要明确说明。

其格式为：

input 端口名1, 端口名2, 端口名n; //输入端口

output 端口名1, 端口名2, 端口名n; //输出端口

inout 端口名1, 端口名2, 端口名n; //输入输出端口

当信号位宽为多位，格式为：

input [n-1:0] 端口名1, 端口名2, 端口名n;



(3) 信号类型声明

- 对模块中所用到的所有信号（包括端口信号、节点信号等）都必须进行数据类型的定义。Verilog语言提供了各种信号类型，分别模拟实际电路中的各种物理连接和物理实体。
- 常用的信号类型有线网型（**wire**）、寄存器型（**reg**）、整数型（**integer**）等，其中输入和双向端口信号不能声明为**reg**型。
- 如果信号的数据类型没有定义，则综合器将其默认为是**wire**型。



(4) 逻辑功能定义

模块中最核心的部分是逻辑功能定义。

定义逻辑功能的几种基本方法：

① 用**assign**持续赋值语句定义

assign语句多用于组合逻辑的赋值，称为持续赋值方式。

② 用**always**过程块定义

always过程语句既可以用来描述组合电路，也可以描述时序电路。

③ 调用元件（元件例化）

调用元件的方法类似于在电路图输入方式下调入图形符号来完成设计，这种方法侧重于电路的结构描述。

例如：

```
vote1 u0  
(.A(A), .B(B), .C(C), .F(F));
```




Test Bench测试仿真文件

```
`timescale 1ns / 1ps
```

```
module vote1_tb;
```

```
  reg A,B,C;
```

```
  wire F;
```

```
  vote1 u0          //例化被测测试模块
```

```
  (.A(A), .B(B), .C(C), .F(F));
```

```
  initial begin      //激励波形描述
```

```
    A=1'b0; B=1'b0; C=1'b0;
```

```
    #100 A=1'b0; B=1'b0; C=1'b1;
```

```
    #100 A=1'b0; B=1'b1; C=1'b0;
```

```
    #100 A=1'b0; B=1'b1; C=1'b1;
```

```
    #100 A=1'b1; B=1'b0; C=1'b0;
```

```
    #100 A=1'b1; B=1'b0; C=1'b1;
```

```
    #100 A=1'b1; B=1'b1; C=1'b0;
```

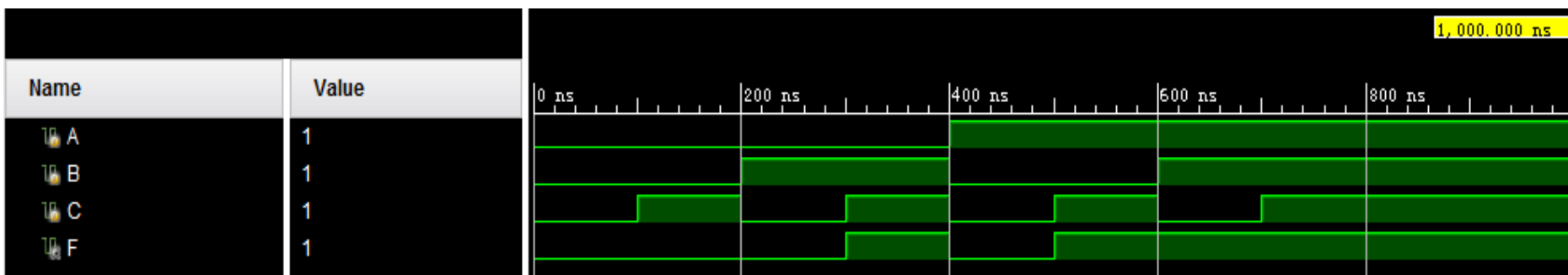
```
    #100 A=1'b1; B=1'b1; C=1'b1;
```

```
  end
```

```
endmodule
```

测试仿真模块说明:

- ▲ **`timescale** 编译器指令在模块说明外部出现，用于定义仿真过程中时间单位和精度的基准；
- ▲ **测试模块只有模块名字**，没有端口列表；
- ▲ **输入信号（激励信号）**必须定义为**reg**型，以保持信号值；
- ▲ **输出信号（显示信号）**必须定义为**wire**型；
- ▲ 一般用**initial**、**always**过程块定义激励信号波形。



仿真波形

■ 参考教材

EDA技术与设计（第2版），电子工业出版社，2024.6

ISBN 978-7-121-48212-0



第3章 小结

- 由基本逻辑门构成的组合电路的分析方法
- 由基本逻辑门构成的组合电路的设计方法
- 组合逻辑电路中的冒险
- 硬件描述语言——Verilog HDL